

Computer Graphics DT082A Lab - 1 Report

Student: Kaan Tekin Öztekin

Date of submission: 23.11.2025

Implementation Process

After establishing a clear structure for creating the cube, assigning RGB values, and applying transformations, the task proceeded more or less as expected. I started with a static cube, but gradually expanded the application by adding colour mapping, shader communication, and a rotation mechanism. Since we continued using the *gasket-2* implementation from the course book, the shader files were kept in a separate directory and the *Common* utilities remained part of the overall architecture. Once the WebGL pipeline (attributes, uniforms, buffers, shaders) was correctly connected, the cube displayed as expected in Fig. 1.

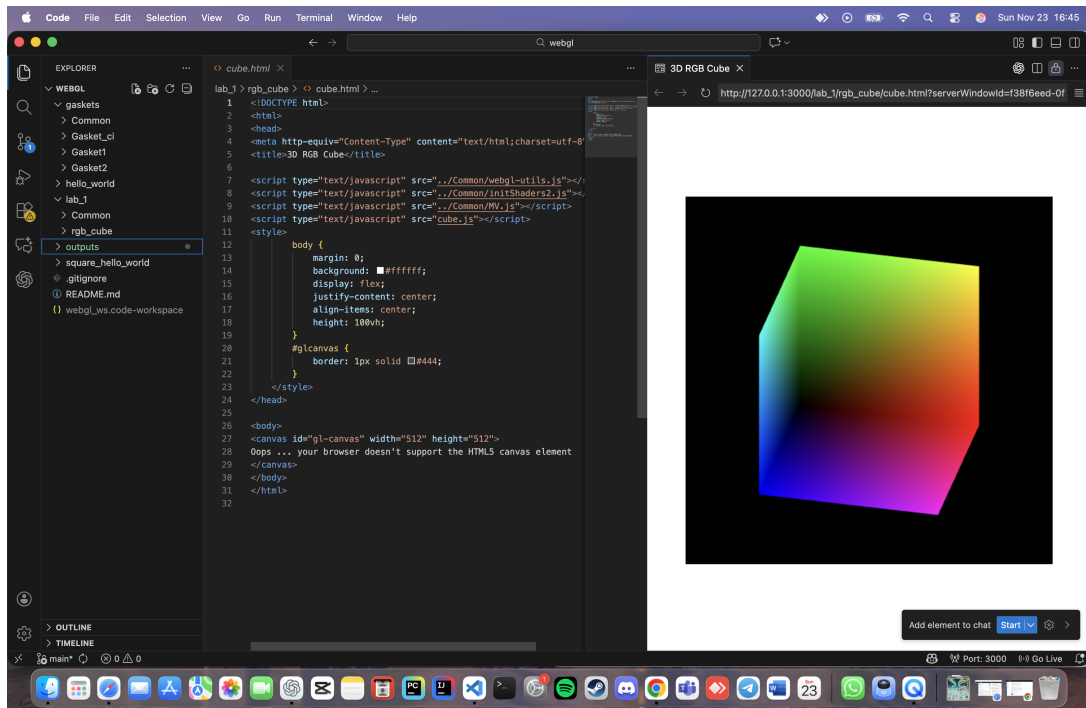


Figure 1: RGB Cube rendered with WebGL.

The full implementation is publicly available at: [GitHub Repository](#).

Obstacles

- One major obstacle was understanding why the cube appeared flat or incorrectly colored at first. I realized the issue was caused by missing or incorrect shader

attribute bindings and not enabling `gl.DEPTH_TEST`.

- At one point the entire canvas turned white, which was caused by a shader compilation error and the `uMVP` uniform not being passed correctly after refactoring the code for animation.
- Another confusion was related to rotation: I initially assumed that rotating vertices would change their coordinates in JavaScript, but learned that transformations occur only inside the vertex shader, leaving the original vertex data unchanged.
- I also questioned why no `lookAt` camera matrix was required, and discovered that the cube is already defined in clip-space, so additional view/projection transformations would contradict the instructions.
- Color smoothness puzzled me; the mid-region of the cube looked gray and “desaturated”. I learned that this is not a rendering bug but an inherent property of the RGB color cube, where interior points naturally converge toward gray due to equal R, G, and B values.

Design Decisions and Architecture

- The cube was represented using eight vertices and six faces, each formed by two triangles (36 vertices). This kept the structure simple.
- RGB values were assigned using a direct linear mapping:
`color = position + 0.5`, following the task requirements.
- Instead of implementing a camera or projection system, only a model rotation matrix was used, as the cube already existed in clip-space.
- Continuous rotation was created by incrementing `theta` inside the render loop and updating the `uMVP` matrix each frame.

Lessons Learned

- I learned the importance of debugging shaders using the browser console; even a small mistake in a uniform or attribute causes the entire canvas to render white.
- I now clearly understand that vertex transformations occur exclusively in the vertex shader and do not alter the original JavaScript arrays.

References

- WebGL Cube Tutorial (YouTube), available online at YouTube.
- WebGL Fundamentals – Cube Maps, available at webglfundamentals.org.
- Drawing a Cube in WebGL (StackOverflow), StackOverflow thread.
- Creating a Simple 3D Scene with Perspective in WebGL, Article.